System Design Supplements

Deep Dives — Distributed Systems Internals

Apple Staff / Principal Engineer Interview Preparation

System Design Series — Printer-Friendly Edition

System Design Supplements — Deep Dives

Distributed Systems Internals for Principal / Staff Engineers

> ***Companion to:*** *systemdesignvol1.md, systemdesignvol2.md, systemdesignvol3.md* ***Level:*** *Staff → Principal Engineer* ***Format:*** *Plain English → Mechanism → Diagram → Code/Config → Apple Interview Tips*

Table of Contents

Chapter 1: Gossip Protocol

Q1 — How do distributed systems like Cassandra discover and monitor nodes without a central coordinator? ■

Plain English First:

Imagine you have 1,000 people in a room and you want everyone to know the same rumor. You don't need a PA system — just have each person whisper it to 3 random neighbors every 30 seconds. Within a few rounds, everyone knows. That's Gossip — epidemic information spreading with no single point of failure.

Why It Exists:

Central heartbeat servers don't scale (the coordinator becomes a bottleneck/SPOF). Gossip spreads information in O(log N) rounds even when nodes fail.

How Gossip Works — 3 Phases:

```
Round 1: Node A gossips to B, C, D  → 4 nodes know
Round 2: B gossips to E,F,G; C gossips to H,I,J; D gossips to K,L,M → 13 nodes know
Round 3: Each of 13 gossips to 3 new → ~40 nodes know
...
O(log N) rounds to reach all N nodes
```

Cassandra Gossip — Concrete Mechanism:

```
Every 1 second, each node:
  1. Picks 1 random LIVE node   → gossip state
  2. Picks 1 random SUSPECTED node → detect failures
  3. Picks 1 random SEED node   → bootstrap rejoins

Message format:
  GossipDigestSyn  → "here's what I know about everyone (node → version)"
  GossipDigestAck  → "here's what I need + here's what you're missing"
  GossipDigestAck2 → "here's what you needed"
```

Failure Detection — Phi Accrual:

Rather than binary up/down, Cassandra uses **Phi Accrual** — a continuous suspicion score.

```
ϕ (phi) = -log10(probability node is still alive given the silence)

ϕ < 8   → alive
ϕ = 8   → suspected (configurable via phi_convict_threshold)
ϕ > 8   → dead

Calculated from: inter-arrival times of heartbeats using exponential distribution
```

```
// Conceptual Phi calculation
class PhiAccrualDetector {
    private final Deque<Long> arrivalIntervals = new ArrayDeque<>();

    void heartbeatReceived(long now) {
        if (!arrivalIntervals.isEmpty()) {
            long interval = now - lastReceived;
            arrivalIntervals.add(interval);
            if (arrivalIntervals.size() > 1000) arrivalIntervals.poll();
        }
        lastReceived = now;
    }

    double phi(long now) {
        long timeSince = now - lastReceived;
        double mean = arrivalIntervals.stream()
            .mapToLong(Long::longValue).average().orElse(1000);
        // CDF of exponential distribution
        double p = Math.exp(-timeSince / mean);
        return -Math.log10(p); // phi
    }
}
```

What Gossip Spreads:

Information	*Purpose*
Node state (UP/DOWN/JOINING/LEAVING)	Membership
Token ranges	Know which node owns which data
Schema version	Detect schema mismatch
DC/rack topology	Replica placement
Load / compaction state	Operational visibility

Convergence Guarantee:

With fanout `f` and `N` nodes, all nodes know the update within `ceil(log_f(N))` rounds. With f=3, N=1000 → 7 rounds. Cassandra gossips every 1 second → full cluster awareness in ~7 seconds.

Anti-Entropy vs Gossip:

- Gossip** = spread membership/metadata fast
- Anti-entropy** = reconcile actual data (uses Merkle trees — see Chapter 2)

> ■ ***Apple Interview Tip:*** *"When designing iCloud Drive file sync across 1M+ devices, Gossip handles which edge nodes are alive. The follow-up is always about failure detection — explain Phi Accrual and why it's better than timeout-based heartbeats."*

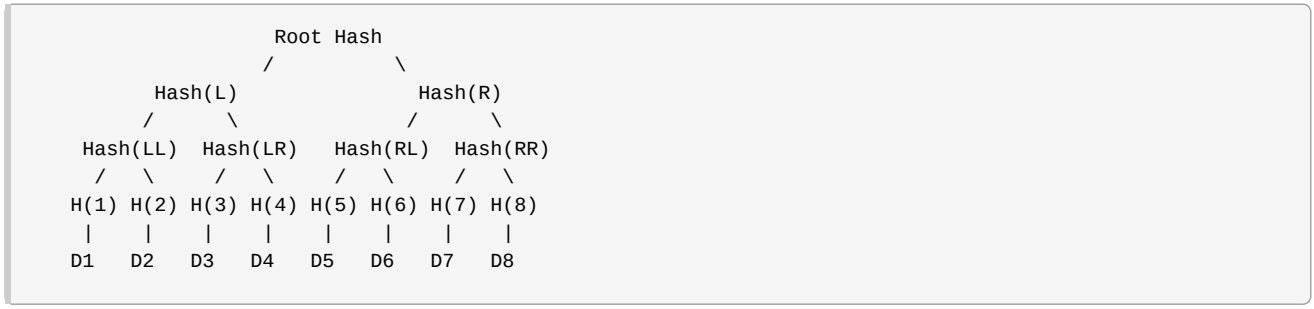
Chapter 2: Merkle Trees

Q2 — How do distributed systems efficiently detect and repair data inconsistencies between replicas? ■

Plain English First:

You have two file cabinets with 1 million files each. You need to find which files differ. Comparing file-by-file takes 1M comparisons. But if you hash folders, then hash-of-hashes, you build a tree where comparing just the top hash tells you instantly if anything differs. Drill down only into mismatched branches to find the exact divergent files. That's a Merkle tree.

Structure:



Key Property: Any change to a leaf bubbles up and changes every ancestor hash. Compare two trees top-down: if roots match → identical; if they differ → descend only into differing branches.

Cassandra Anti-Entropy Repair:

```
1. Node A builds Merkle tree over its token range
2. Node B builds Merkle tree over the same range
3. Compare trees at root → find first mismatching subtree
4. Drill down to leaf level → O(log N) comparisons
5. Stream only the mismatched rows to sync

Without Merkle: compare all 10M rows
With Merkle:   compare log2(10M) ≈ 23 hashes, then stream just the bad rows
```

```
// Simplified Merkle tree
class MerkleTree {
    private final byte[][] leaves;
    private final byte[][] tree;
    private final int n; // must be power of 2

    MerkleTree(List<byte[]> data) {
        this.n = nextPowerOf2(data.size());
        this.leaves = new byte[n][];
        this.tree = new byte[2 * n][];

        // Set leaves
        for (int i = 0; i < data.size(); i++) {
            leaves[i] = sha256(data.get(i));
            tree[n + i] = leaves[i];
        }
        // Fill empty leaves with hash of empty
        for (int i = data.size(); i < n; i++) {
            tree[n + i] = sha256(new byte[0]);
        }
        // Build internal nodes bottom-up
        for (int i = n - 1; i >= 1; i--) {
            tree[i] = sha256(concat(tree[2*i], tree[2*i+1]));
        }
    }

    byte[] root() { return tree[1]; }

    // Returns list of (index, hash) pairs that differ
    List<Integer> diff(MerkleTree other) {
        List<Integer> diffLeaves = new ArrayList<>();
        diffSubtree(1, other, diffLeaves);
        return diffLeaves;
    }

    private void diffSubtree(int node, MerkleTree other, List<Integer> result) {
        if (Arrays.equals(this.tree[node], other.tree[node])) return; // subtree matches
        if (node >= n) { // leaf
            result.add(node - n);
            return;
        }
        diffSubtree(2 * node, other, result);     // left
        diffSubtree(2 * node + 1, other, result); // right
    }
}
```

Use Cases Beyond Cassandra:

System	*Use*
Bitcoin/Ethereum	Block = Merkle root of all txs. Light clients verify a single tx in O(log N) without downloading the full block
Git	Every commit is a Merkle DAG of tree objects. `git diff` traverses only changed subtrees

System	*Use*
ZFS / btrfs	Filesystem integrity — detect silent corruption
DynamoDB	Replica reconciliation across AZs
IPFS	Content-addressed storage — hash IS the address

Blockchain Merkle Proof (SPV):

```
Full block has 10,000 transactions.
Light client wants to verify Tx #5,000 is included:
  → Server sends: Tx#5000, sibling hashes at each level (log2(10000) ≈ 14 hashes)
  → Client recomputes root → matches block header → proven
  Cost: 14 hashes instead of 10,000 transactions downloaded
```

> ■ ***Apple Interview Tip:*** *"For iCloud Photo Library with billions of photos across replicas, Merkle trees let us find which photos are missing on a replica in O(log N) comparisons vs O(N) full scan. Follow-up: how do you handle tree rebalancing when items are inserted?"*

Chapter 3: Vector Clocks & Causality

Q3 — How do distributed systems track causality and detect conflicting writes without a global clock? ■

Plain English First:

In a chat app, Alice sends "are you coming?" and Bob replies "yes!" — you know Bob's message happened *after* Alice's. But in a distributed system, wall clocks disagree (clock skew). Vector clocks give each node its own logical counter, and together they form a version vector that can determine if event A happened before, after, or *concurrently* (potential conflict) with event B.

Structure:

Each node maintains a vector `[v1, v2, ..., vN]` where vi = number of events at node i that the current node knows about.

```
System: 3 nodes A, B, C
Initial: A=[0,0,0], B=[0,0,0], C=[0,0,0]

A writes X:   A=[1,0,0]  → message sent with VC=[1,0,0]
B receives:   B=[1,1,0]  → B increments own, merges received
B writes Y:   B=[1,2,0]  → Y happened AFTER X (VC dominates)
C writes Z:   C=[0,0,1]  → concurrent with B's write! (neither dominates)
```

Happens-Before Rules:

```
VC(a) < VC(b)  iff  ∀i: VC(a)[i] ≤ VC(b)[i]  AND  ∃j: VC(a)[j] < VC(b)[j]
  → a happened before b (causal order established)

VC(a) || VC(b)  iff  neither dominates the other
  → concurrent events → potential conflict → need resolution
```

```
class VectorClock {
    private final Map<String, Integer> clock;
    private final String nodeId;

    VectorClock(String nodeId) {
        this.nodeId = nodeId;
        this.clock = new HashMap<>();
    }

    // Increment own counter on local event
    void tick() {
        clock.merge(nodeId, 1, Integer::sum);
    }

    // Merge on message receive: take max of each component, then tick own
    void receive(Map<String, Integer> incoming) {
        incoming.forEach((node, count) ->
            clock.merge(node, count, Math::max));
        tick();
    }

    // Returns -1 (before), 0 (concurrent), 1 (after)
    int compareTo(VectorClock other) {
        boolean lessThanOrEqual = true;
        boolean greaterThanOrEqual = true;

        Set<String> allNodes = new HashSet<>();
        allNodes.addAll(this.clock.keySet());
        allNodes.addAll(other.clock.keySet());

        for (String node : allNodes) {
            int mine = this.clock.getOrDefault(node, 0);
            int theirs = other.clock.getOrDefault(node, 0);
            if (mine < theirs) greaterThanOrEqual = false;
            if (mine > theirs) lessThanOrEqual = false;
        }

        if (lessThanOrEqual && !greaterThanOrEqual) return -1; // this < other
        if (greaterThanOrEqual && !lessThanOrEqual) return 1;  // this > other
        return 0; // concurrent
    }

    Map<String, Integer> snapshot() { return Collections.unmodifiableMap(clock); }
}
```

DynamoDB — Version Vectors in Practice:

DynamoDB uses a variant called *version vectors* (server-side vector clocks). When two writes conflict (concurrent), DynamoDB returns both versions and the client must reconcile (or use Last-Write-Wins if

configured).

```
Client writes item → DynamoDB stores with VC [S1:3, S2:1]
Network partition → two nodes accept concurrent writes:
  Node 1 → VC [S1:4, S2:1]  (user updated name)
  Node 2 → VC [S1:3, S2:2]  (user updated email)

Neither dominates → CONFLICT → DynamoDB returns BOTH
Client code merges: { name from v1, email from v2 } → write reconciled version
```

Vector Clocks vs Lamport Timestamps:

	Lamport	***Vector Clock***
Size	O(1)	O(N nodes)
Detects concurrent?	No (only ordering)	Yes
Causality?	Partial (LC(a)<LC(b) doesn't mean a→b)	Full
Use case	Total ordering for logs	Conflict detection

Real-World Conflict Resolution Strategies:

1. **Last Write Wins (LWW):** Cassandra default — highest timestamp wins. Risk: losing data on clock skew.
2. **Multi-Value (MV):** Riak — return all conflicting versions, client merges. Correct but complex.
3. **CRDTs:** Design data structures where concurrent ops always merge deterministically (grow-only counters, sets, etc.)
4. **Application-level merge:** Shopping cart — union of items (Amazon Dynamo paper).

> ■ ***Apple Interview Tip:*** *"For Apple Pay, concurrent writes must never lose money — vector clocks detect conflicts, and pessimistic locking or saga compensation handles resolution. Never use LWW for financial data."*

Chapter 4: Paxos Algorithm

Q4 — How does Paxos achieve consensus, and how does it differ from Raft? ■

Plain English First:

10 generals must agree on whether to attack tomorrow. They communicate by letter (may be lost). Paxos is a protocol where one general (the Proposer) gets a majority to agree on a value, even if some letters are lost. The key insight: a value is "chosen" once a majority (quorum) accepts it — even if not everyone knows it yet.

Paxos Roles:

- Proposer** — proposes a value (usually the leader)
- - **Acceptor** — votes to accept proposals (stores `promised_n` and `accepted_n, accepted_v`)
- Learner** — learns what was decided (usually the client)

Two Phases:

```
=== PHASE 1: PREPARE (Leader Election) ===

Proposer sends: Prepare(n)  [n = unique proposal number]

Acceptors respond:
  IF n > promised_n:
    promised_n = n
    reply Promise(n, accepted_n, accepted_v)  // "I won't accept anything < n"
  ELSE:
    Nack  // ignore

Proposer collects majority (quorum) of Promises
  If any acceptor already accepted a value → Proposer MUST use that value (safety)
  Otherwise → Proposer can use its own value (freedom)

=== PHASE 2: ACCEPT ===

Proposer sends: Accept(n, v)  [v = chosen value]

Acceptors respond:
  IF n >= promised_n:
    accepted_n = n, accepted_v = v
    reply Accepted(n, v)
  ELSE:
    Nack

Proposer collects majority Accepted → VALUE IS CHOSEN ✓
Proposer notifies Learners
```

Concrete Example:

```
5 acceptors: A1..A5. Quorum = 3

Round 1 — Proposer P1 (n=1):
  Prepare(1)  → A1,A2,A3 promise (A4,A5 don't respond — network)
  No prior accepted values → P1 proposes v="attack at dawn"
  Accept(1,"attack at dawn") → A1,A2,A3 accept ✓ → CHOSEN

But P2 (n=2) starts concurrently, gets Prepare responses from A3,A4,A5:
  A3 reports: accepted_n=1, accepted_v="attack at dawn"
  → P2 MUST propose "attack at dawn" too (safety constraint)
  → Both proposers converge to the same value ✓
```

Paxos Safety Guarantee:

If a value v is chosen (accepted by a quorum), then any future quorum MUST include at least one acceptor that saw v. Since quorums overlap by at least 1, any new proposer will learn v and be forced to propose v.

Two different values can never both be chosen.

Multi-Paxos (Practical Paxos):

Basic Paxos agrees on ONE value. Real systems use Multi-Paxos for a log of decisions:

- Phase 1 is amortized: run once for a leader term → leader has "pre-approval" for slot sequence
- Phase 2 runs for each log slot
- This is essentially what Raft is a cleaner formulation of

Paxos vs Raft Comparison:

Aspect	*Paxos*	*Raft*
Conceptual model	Single-decree → Multi	Log replication
Leader election	Implicit (highest n wins)	Explicit term-based
Log consistency	Complex (holes, out-of-order)	Leader log is truth
Membership change	Painful	Joint consensus
Implementation	Google Chubby, Zookeeper	etcd, CockroachDB, TiKV
Understandability	"notoriously difficult"	Designed for understandability
Used in prod	Spanner, Chubby, Cassandra HLC	Kubernetes (etcd), CockroachDB

Paxos Liveness Problem — Dueling Proposers:

```
P1 prepares n=1  → A1,A2,A3 promise
P2 prepares n=2  → A1,A2,A3 promise (revokes P1's promise)
P1 prepares n=3  → A1,A2,A3 promise (revokes P2's)
P2 prepares n=4  → ...
→ Infinite loop, no progress!

Solution: Randomized backoff + Distinguished Proposer (leader lease)
```

Google Chubby (Production Paxos):

- Chubby is a distributed lock service built on Paxos
- Used by GFS (master election), BigTable (tablet master), MapReduce (job coordination)
- 5 replicas, quorum=3, one elected master via Paxos
- Master holds a lease (typically 12s) — clients cache master identity

> ■ ***Apple Interview Tip:*** *"Interviewers don't expect full Paxos code. Know: two phases, quorum, why a new proposer must adopt the highest accepted value (safety), and the liveness problem. Compare to Raft — Raft is Paxos made understandable."*

Chapter 5: Skip Lists Internals

Q5 — How do Redis sorted sets and database MemTables achieve O(log N) operations without rebalancing? ■

Plain English First:

A sorted linked list has O(N) search. A balanced BST has O(log N) but complex rotations on insert. A skip list adds "express lanes" — higher-level pointers that let you skip over many nodes. It achieves O(log N) average-case performance with simple randomized insertion and no rotations.

Structure:

```
Level 3:  1 ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■→ 50 ■■■■■■■■■■■■■■→ null
Level 2:  1 ■■■■■■■■■■■■→ 20 ■■■■■■■■■■■■■■■■■■→ 50 → 70 ■■■■■■■→ null
Level 1:  1 → 10 ■■■■■→ 20 → 30 ■■■■■■■■■■■■→ 50 → 70 → 90 → null
Level 0:  1 → 10 → 15→ 20 → 30 → 40 → 45→ 50 → 70 → 90 → null
          (base linked list — all elements)
```

Search Algorithm:

```
Search for 45:
Level 3: 1 < 45? Yes → jump. Next = null → drop to Level 2
Level 2: 1 < 45? Yes → jump. 20 < 45? Yes → jump. 50 > 45 → drop to Level 1
Level 1: 20 < 45? Yes → jump. 30 < 45? Yes → jump. 50 > 45 → drop to Level 0
Level 0: 30 < 45? Yes → jump. 40 < 45? Yes → jump. 45 = found! ✓

Comparisons: ~log N (3 levels, skipped most nodes)
```

Node Structure:

```
class SkipListNode<K extends Comparable<K>, V> {
    K key;
    V value;
    SkipListNode<K, V>[] next; // array of forward pointers, one per level

    @SuppressWarnings("unchecked")
    SkipListNode(K key, V value, int level) {
        this.key = key;
        this.value = value;
        this.next = new SkipListNode[level];
    }
}

class SkipList<K extends Comparable<K>, V> {
    private static final int MAX_LEVEL = 32;
    private static final double P = 0.5; // probability of level promotion
    private final SkipListNode<K, V> head;
    private int currentMaxLevel = 1;
    private final Random random = new Random();

    @SuppressWarnings("unchecked")
    SkipList() {
        head = new SkipListNode<>(null, null, MAX_LEVEL);
    }

    // Coin flip: each node gets level L with probability P^(L-1)
    private int randomLevel() {
        int level = 1;
        while (random.nextDouble() < P && level < MAX_LEVEL) level++;
        return level;
    }

    V search(K key) {
        SkipListNode<K, V> cur = head;
        for (int i = currentMaxLevel - 1; i >= 0; i--) {
            while (cur.next[i] != null && cur.next[i].key.compareTo(key) < 0) {
                cur = cur.next[i]; // advance at this level
            }
        }
        cur = cur.next[0]; // base level
        return (cur != null && cur.key.compareTo(key) == 0) ? cur.value : null;
    }

    void insert(K key, V value) {
        // Track predecessors at each level (update array)
        @SuppressWarnings("unchecked")
        SkipListNode<K, V>[] update = new SkipListNode[MAX_LEVEL];
        SkipListNode<K, V> cur = head;

        for (int i = currentMaxLevel - 1; i >= 0; i--) {
```

```
            while (cur.next[i] != null && cur.next[i].key.compareTo(key) < 0) {
                cur = cur.next[i];
            }
            update[i] = cur;
        }

        int newLevel = randomLevel();
        if (newLevel > currentMaxLevel) {
            for (int i = currentMaxLevel; i < newLevel; i++) update[i] = head;
            currentMaxLevel = newLevel;
        }

        SkipListNode<K, V> newNode = new SkipListNode<>(key, value, newLevel);
        for (int i = 0; i < newLevel; i++) {
            newNode.next[i] = update[i].next[i];
            update[i].next[i] = newNode;
        }
    }

    // Range query: O(log N + K) where K = result size — key advantage over hash maps
    List<V> range(K from, K to) {
        SkipListNode<K, V> cur = head;
        for (int i = currentMaxLevel - 1; i >= 0; i--) {
            while (cur.next[i] != null && cur.next[i].key.compareTo(from) < 0) {
                cur = cur.next[i];
            }
        }
        cur = cur.next[0];
        List<V> result = new ArrayList<>();
        while (cur != null && cur.key.compareTo(to) <= 0) {
            result.add(cur.value);
            cur = cur.next[0];
        }
        return result;
    }
}
```

Complexity Analysis:

Operation	*Average*	*Worst*
Search	O(log N)	O(N)
Insert	O(log N)	O(N)
Delete	O(log N)	O(N)
Range query	O(log N + K)	O(N)
Space	O(N log N)	O(N log N)

Worst case is rare (exponentially unlikely with random promotion). Expected height = O(log N).

Why Redis Uses Skip Lists for Sorted Sets (ZSET):

- Range queries: `ZRANGEBYSCORE`, `ZRANGE` — O(log N + K) — skip lists excel at this

- Simpler concurrent implementation than balanced BST (only local pointer changes)
- Alternative considered: AVL/Red-Black tree — complex rotations, worse cache locality
- Redis ZSET uses skip list + hash map: O(1) score lookup by member, O(log N) rank queries

Why LevelDB/RocksDB MemTable Uses Skip List:

- MemTable is in-memory, needs O(log N) insert/lookup + sequential scan (for SSTable flush)
- Skip list's base level is a sorted linked list → perfect for sequential flush to SSTable
- Lock-free skip list variants exist for concurrent MemTable writes

> ■ ***Apple Interview Tip:*** *"If asked to design a leaderboard (Apple Arcade scores), reach for sorted set semantics. Explain skip list as the backing structure — O(log N) rank updates, O(log N + K) range queries for 'top 100 players'. Compare to sorted array (O(N) insert) or heap (no range queries)."*

Chapter 6: Kafka Streams & Windowing

Q6 — How do streaming systems compute aggregations over time windows like "messages per minute" or "fraud alerts in sliding 5-minute window"? ■

Plain English First:

A stream is an infinite sequence of events. You can't aggregate "all time" — you slice time into windows. Three window types solve different problems:

- Tumbling:** Non-overlapping fixed slices (hourly sales totals)
- Sliding:** Overlapping windows (5-min fraud detection — any 5-min period)
- Session:** Variable-length bursts of activity (user session analytics)

Kafka Streams Architecture:

```
Kafka Topics (input)
      ↓
KStream / KTable (stream processing abstraction)
      ↓
Windowed Aggregation
      ↓
State Store (RocksDB-backed, changelog topic for fault tolerance)
      ↓
Output Topic / Interactive Query
```

Tumbling Windows — Non-Overlapping, Fixed Size:

```
Events: |--e1--e2--|--e3--e4--e5--|--e6--|
Time:   |   W1     |     W2       |  W3  |
        0         10             20     30 (seconds)

Use case: "Count orders per 10-second window for dashboard"
```

```
// Kafka Streams — Count events per tumbling 1-minute window
StreamsBuilder builder = new StreamsBuilder();

KStream<String, OrderEvent> orders = builder.stream("orders");

orders
    .groupByKey()
    .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(1)))
    .count(Materialized.<String, Long, WindowStore<Bytes, byte[]>>as("order-counts")
        .withValueSerde(Serdes.Long()))
    .toStream()
    .map((windowedKey, count) -> {
        String key = windowedKey.key();
        long windowStart = windowedKey.window().start();
        long windowEnd = windowedKey.window().end();
        return new KeyValue<>(key, new WindowResult(key, count, windowStart, windowEnd));
    })
    .to("order-counts-output");
```

Sliding Windows — Overlapping, Fixed Size:

```
Events: e1(t=1), e2(t=3), e3(t=6), e4(t=8), e5(t=12)
Window size = 5 sec, every event creates a window centered on it

e1's window: [0, 5]   → contains e1, e2
e2's window: [0, 5]   → contains e1, e2
e3's window: [3, 8]   → contains e2, e3, e4
e4's window: [4, 9]   → contains e3, e4
e5's window: [9, 14]  → contains e5

Use case: "Fraud detection — if user has >3 failed logins in any 5-minute window"
```

```
// Sliding window — fraud detection
KStream<String, LoginAttempt> logins = builder.stream("login-attempts");

logins
    .filter((userId, attempt) -> !attempt.isSuccess())
    .groupByKey()
    .windowedBy(SlidingWindows.ofTimeDifferenceWithNoGrace(Duration.ofMinutes(5)))
    .count()
    .toStream()
    .filter((windowedKey, count) -> count >= 3) // threshold
    .map((windowedKey, count) ->
        new KeyValue<>(windowedKey.key(), new FraudAlert(windowedKey.key(), count)))
    .to("fraud-alerts");
```

Session Windows — Activity-Based, Variable Length:

```
User activity: click(t=1) click(t=3) click(t=4) [gap=8s] click(t=15) click(t=16)
Inactivity gap = 5 seconds

Session 1: [1, 4]  → 3 events (gap < 5s so merged)
Session 2: [15,16] → 2 events (new session after 8s gap)

Use case: "Average session length in Apple Music, bounce rate analytics"
```

```
// Session window — user engagement analytics
KStream<String, PageView> pageViews = builder.stream("page-views");

pageViews
    .groupByKey()
    .windowedBy(SessionWindows.ofInactivityGapWithNoGrace(Duration.ofSeconds(30)))
    .aggregate(
        () -> new SessionAggregate(), // initializer
        (userId, view, agg) -> agg.addView(view), // aggregator
        (userId, agg1, agg2) -> agg1.merge(agg2), // session merger (when sessions join)
        Materialized.with(Serdes.String(), sessionAggregateSerde)
    )
    .toStream()
    .map((windowedKey, agg) -> {
        long duration = windowedKey.window().end() - windowedKey.window().start();
        return new KeyValue<>(windowedKey.key(), new SessionMetric(duration, agg.pageCount()));
    })
    .to("session-metrics");
```

Late Data & Watermarks:

Events arrive out of order (network delays). Kafka Streams handles this via **grace period**:

```
// Allow events up to 30 seconds late
TimeWindows.ofSizeAndGrace(Duration.ofMinutes(1), Duration.ofSeconds(30))

// Without grace: late events dropped silently
// With grace:    window stays open for 30 extra seconds, then closes
```

State Store Fault Tolerance:

```
State store (RocksDB)  ←→  Changelog topic (Kafka)
     ↑                          ↑
  Fast local reads         Replicated for recovery

If task fails → restart → replay changelog → restore state
```

Window Types Comparison:

Type	*Overlap*	*Size*	*Use Case*
Tumbling	None	Fixed	Hourly billing, daily reports

Type	*Overlap*	*Size*	*Use Case*
Hopping	Yes	Fixed (hop < window)	Metrics every 1min over 5min
Sliding	Yes	Fixed, event-driven	Fraud, anomaly detection
Session	Yes	Variable	User analytics, IoT device activity

Interactive Queries — Query State Stores Directly:

```
// Query the windowed state store without going through output topic
ReadOnlyWindowStore<String, Long> windowStore =
    streams.store(StoreQueryParameters.fromNameAndType(
        "order-counts",
        QueryableStoreTypes.windowStore()));

// Get count for a specific user in last hour
Instant now = Instant.now();
WindowStoreIterator<Long> iterator =
    windowStore.fetch("user-123", now.minus(Duration.ofHours(1)), now);
while (iterator.hasNext()) {
    KeyValue<Long, Long> next = iterator.next();
    System.out.println("Window at " + next.key + ": " + next.value);
}
```

> ■ ***Apple Interview Tip:*** *"For Apple Music trending songs, use a hopping window: 1-hour window, advance every 5 minutes. This gives 'trending in the last hour' updated frequently. Session windows for user listening session analytics. Always discuss late data handling — specify grace periods."*

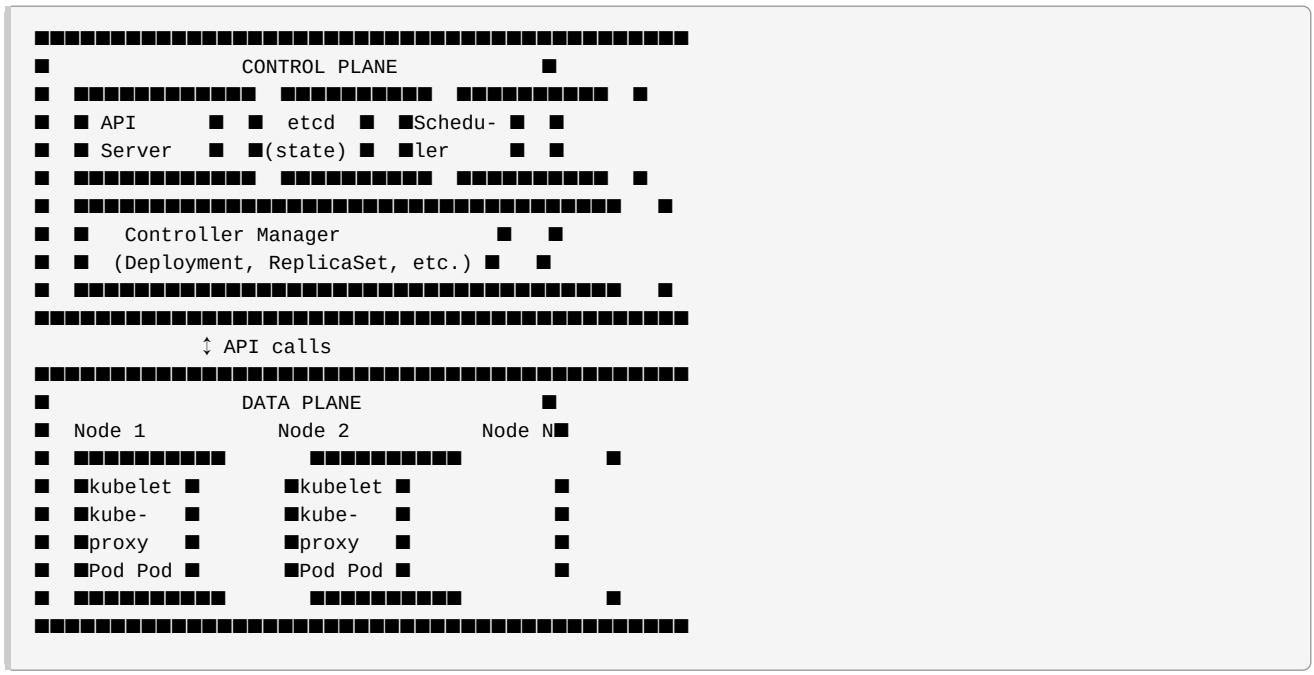
Chapter 7: Kubernetes & Container Orchestration

Q7 — How does Kubernetes schedule workloads and recover from failures at scale? ■

Plain English First:

You have 1,000 servers and 10,000 containers to run. Kubernetes is the operating system for this datacenter — it decides where to run containers, restarts them if they crash, scales them up/down based on load, and manages networking between them.

Core Architecture:



Key Components:

Component	*Role*
etcd	Consistent distributed KV store — all cluster state
API Server	REST gateway to etcd. All components talk through it
Scheduler	Assigns Pods to Nodes based on resources/constraints
Controller Manager	Reconciliation loops: "desired state → actual state"
kubelet	Agent on each node: starts/stops containers via container runtime
kube-proxy	Manages iptables/IPVS rules for Service → Pod routing

Scheduler — How It Picks a Node:

```
Step 1: FILTERING (hard constraints)
  - Node has enough CPU/Memory?
  - NodeSelector matches?
  - Taint/Toleration allows?
  - PodAffinity satisfied?
  → Filters out ineligible nodes

Step 2: SCORING (soft preferences)
  - Least-requested CPU: prefer less loaded nodes
  - Image locality: node already has the container image
  - Inter-pod affinity: prefer nodes near related pods
  → Each node gets a score 0-100

Step 3: BINDING
  → Highest-scored node selected
  → API Server writes Pod.spec.nodeName = "selected-node"
  → kubelet on that node sees unbound pod → starts container
```

Reconciliation Loop — The Core Pattern:

```
// Every K8s controller runs a reconcile loop (conceptually):
while (true) {
    desiredState = etcd.get("deployment/my-app");  // what user wants
    actualState  = runtime.getPods("my-app");       // what exists

    if (actualState.count < desiredState.replicas) {
        createPod();  // scale up
    } else if (actualState.count > desiredState.replicas) {
        deletePod();  // scale down
    }
    // If equal → do nothing
    sleep(resyncPeriod);
}
```

Pod Failure Recovery:

```
kubelet → detects container crash
       → respects restartPolicy: Always/OnFailure/Never
       → restarts with exponential backoff: 10s, 20s, 40s, 80s, 160s, 5min (capped)

ReplicaSet Controller → watches pods
                     → Pod status = Failed → creates replacement Pod
                     → Scheduler → places new pod on healthy node

Node failure:
  kubelet stops heartbeating to API server
  Node Controller waits node-monitor-grace-period (40s default)
  Node marked NotReady
  After pod-eviction-timeout (5min): pods on node marked for eviction
  New pods scheduled on remaining nodes
```

Services — Stable Networking:

```
apiVersion: v1
kind: Service
metadata:
  name: my-api
spec:
  selector:
    app: my-api           # Routes to pods with this label
  ports:
  - port: 80              # Service port
    targetPort: 8080      # Pod port
  type: ClusterIP         # Internal only (LoadBalancer for external)
```

```
Client → Service IP (virtual, stable) → kube-proxy → Pod IP (ephemeral)
kube-proxy uses iptables/IPVS to NAT traffic to one of the healthy pods
This is why pod IPs can change but Service IP stays constant
```

Horizontal Pod Autoscaler (HPA):

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    kind: Deployment
    name: my-api
  minReplicas: 2
  maxReplicas: 50
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70   # scale when avg CPU > 70%
  - type: External
    external:
      metric:
        name: kafka_consumer_lag  # custom metric via Prometheus adapter
      target:
        type: AverageValue
        averageValue: "1000"
```

Resource Requests vs Limits:

```
resources:
  requests:
    cpu: "500m"    # Scheduler uses this for placement (0.5 CPU guaranteed)
    memory: "256Mi"
  limits:
    cpu: "2000m"   # Can burst to 2 CPU (throttled if exceeded)
    memory: "512Mi" # If exceeded → OOMKilled
```

Design: Running a Stateful Service on Kubernetes (e.g., PostgreSQL):

```
apiVersion: apps/v1
kind: StatefulSet   # NOT Deployment — preserves identity and storage
spec:
  serviceName: postgres
  replicas: 3
  volumeClaimTemplates:
  - metadata:
      name: postgres-data
    spec:
      storageClassName: fast-ssd
      accessModes: [ReadWriteOnce]
      resources:
        requests:
          storage: 100Gi
  # Each pod gets stable DNS: postgres-0.postgres, postgres-1.postgres
  # Pod postgres-0 gets PVC postgres-data-postgres-0 (never deleted on pod restart)
```

Production Concerns:

Concern	*Solution*
Zero-downtime deploy	RollingUpdate strategy + readiness probes
Node failure	PodDisruptionBudget (min available replicas)
Config management	ConfigMap + Secret (ideally sealed with Sealed Secrets)
Service discovery	CoreDNS: `service.namespace.svc.cluster.local`
Storage	PersistentVolume + StorageClass (EBS, GCE PD, etc.)
Multi-tenancy	Namespaces + RBAC + NetworkPolicy + LimitRange
Observability	Prometheus metrics, Jaeger tracing, EFK logs

■ ***Apple Interview Tip:*** *"Apple runs thousands of microservices for iCloud, App Store, Apple Pay. Know the scheduler filter→score→bind flow, StatefulSet vs Deployment trade-offs, and how HPA integrates with custom metrics (Kafka lag, request queue depth). Pod disruption budgets are critical for Apple's zero-downtime requirements."*

Chapter 8: DNS Internals

Q8 — How does DNS resolution work, and how do systems use DNS for load balancing, failover, and geographic routing? ■

Plain English First:

DNS is the internet's phone book. You ask "what's the IP for api.apple.com?" and a chain of servers answer. The magic is in the hierarchy — no single server knows everything, but together they can resolve any name in the world. Modern DNS is also used for load balancing, blue/green deployments, and routing users to the nearest datacenter.

Resolution Process — Full Recursive Query:

```
Browser: What's api.apple.com?
    ■
    ▼
1. OS Cache: Do I have it cached? (check /etc/hosts, then local cache)
   → No (or expired)
    ■
    ▼
2. Recursive Resolver (your ISP or 8.8.8.8/1.1.1.1)
   → Do I have it cached? → No
    ■
    ▼
3. Root Name Server (13 sets: a.root-servers.net .. m.root-servers.net)
   Query: "What nameserver handles .com?"
   Answer: "Ask g.gtld-servers.net" (the .com TLD server)
    ■
    ▼
4. .com TLD Name Server
   Query: "What nameserver handles apple.com?"
   Answer: "Ask ns1.apple.com" (apple's authoritative NS)
    ■
    ▼
5. Apple's Authoritative Name Server (ns1.apple.com)
   Query: "What's api.apple.com?"
   Answer: "52.1.2.3" (A record) with TTL=60
    ■
    ▼
6. Resolver caches the answer (TTL=60s), returns to browser
7. Browser connects to 52.1.2.3
```

DNS Record Types:

Record	*Purpose*	*Example*
A	IPv4 address	api.apple.com → 17.32.123.45
AAAA	IPv6 address	api.apple.com → 2001:db8::1
CNAME	Alias to another name	www.apple.com → apple.com
MX	Mail server	apple.com → mx1.apple.com

Record	*Purpose*	*Example*
TXT	Arbitrary text (SPF, DKIM, verification)	"v=spf1 include:..."
NS	Authoritative nameservers	apple.com → ns1.apple.com
SOA	Start of Authority (zone metadata)	serial, refresh, retry, expire
SRV	Service locator (port + priority)	*http.*tcp.apple.com → host:port
PTR	Reverse DNS (IP → name)	45.123.32.17.in-addr.arpa → api.apple.com

TTL — The Cache Control Knob:

```
TTL = 300  → clients cache for 5 minutes → 5min propagation delay on change
TTL = 60   → 1 minute cache → near-realtime failover, but 60x more DNS queries
TTL = 3600 → 1 hour cache → low load on DNS servers, slow failover

Before a planned deployment:
1. Reduce TTL to 60s (wait for existing caches to expire — up to old TTL hours)
2. Perform change
3. Set TTL back to 300s/3600s

"TTL lowering lead time" = previous TTL duration before you start
```

DNS for Load Balancing:

1. Round-Robin DNS:

```
api.apple.com → [17.1.1.1, 17.1.1.2, 17.1.1.3]
Resolver returns all IPs, rotates the order
Client picks first IP (usually)
Problem: Doesn't know if server is healthy — no health checks
```

2. Weighted DNS:

```
api.apple.com:
  17.1.1.1  weight=70  (primary datacenter)
  17.1.1.2  weight=30  (secondary)
Used for blue/green: gradually shift 10% → 50% → 100%
```

3. Failover DNS:

```
Primary:   17.1.1.1 (health-checked by DNS provider)
Secondary: 17.2.2.1 (activated if primary fails health check)
TTL=30s during failover scenarios
Route53, NS1, Cloudflare all support health-checked failover
```

GeoDNS — Route Users to Nearest Datacenter:

```
User from Europe queries api.apple.com:
  DNS server detects source IP is in EU
  Returns: 192.168.1.1 (EU datacenter)

User from Asia queries api.apple.com:
  Returns: 192.168.2.1 (AP datacenter)

Implementation: DNS server has GeoIP database
Providers: Route53 Geolocation, Cloudflare, Akamai
Latency improvement: 50-200ms by routing to nearby region
```

Anycast DNS:

```
Same IP address announced from multiple physical locations
BGP routing directs each user to nearest announce point

13.0.0.1 announced from: New York, London, Tokyo, Sydney, São Paulo
User's packet follows BGP "shortest path" to nearest server
Used by: Root DNS servers (13 IPs, hundreds of physical servers)
         Cloudflare 1.1.1.1, Google 8.8.8.8
```

Split-Horizon DNS:

```
Internal DNS:  api.apple.com → 10.0.0.100 (private IP, internal load balancer)
External DNS:  api.apple.com → 17.1.1.1   (public IP, external load balancer)

Corporate network uses internal resolver → gets private route
Internet users use public resolver → get public IP
```

DNS Security:

Attack	*Defense*
Cache poisoning	DNSSEC (sign records with PKI)
DDoS on resolvers	Anycast disperses traffic
DNS hijacking	DNSSEC + DoH/DoT
DNS-based exfiltration	DNS firewall (RPZ), anomaly detection
NXDOMAIN amplification	Rate limiting, Response Rate Limiting (RRL)

DNS over HTTPS (DoH) / DNS over TLS (DoT):

```
Traditional DNS: plaintext UDP port 53
  → ISP can see every domain you query
  → Susceptible to MITM/injection

DoH: DNS queries inside HTTPS to resolver (port 443)
DoT: DNS queries inside TLS (port 853)
→ Encrypted, harder to intercept or inject
→ Cloudflare 1.1.1.1, Google 8.8.8.8 both support DoH/DoT
```

DNS in Kubernetes (CoreDNS):

```
Pod queries: my-service.my-namespace.svc.cluster.local
CoreDNS (running as pods in kube-system) answers:
  → Returns ClusterIP of the Service
  → kube-proxy then routes to actual pods

Search domains in /etc/resolv.conf:
  search my-namespace.svc.cluster.local svc.cluster.local cluster.local
  → So pod can just call "my-service" and OS appends search domains
```

> ■ ***Apple Interview Tip:*** *"iCloud.com serves billions of requests. GeoDNS routes users to nearest PoP, anycast handles DDoS, and low-TTL health-checked failover achieves sub-minute failover. Always ask: what happens during DNS propagation? Answer: negative caching, TTL expiry, regional resolvers may lag."*

Chapter 9: CDN Internals

Q9 — How do CDNs like Akamai and Cloudflare serve content at global scale, and how do you design for CDN efficiency? ■

Plain English First:

A CDN is a network of servers (Points of Presence — PoPs) distributed worldwide. Instead of every user downloading from your origin server in Cupertino, they download from a server 10ms away in their city. This reduces latency by 10-100x, dramatically reduces load on your origin, and absorbs DDoS attacks.

CDN Architecture:

```
User (Tokyo) → CDN Edge (Tokyo PoP) → Cache HIT → respond in 5ms
                                     → Cache MISS → Origin Shield (Singapore)
                                                  → Cache HIT → 50ms
                                                  → Cache MISS → Origin (Cupertino) → 200ms
```

Cache Hierarchy:

```
Layer 1: Edge PoPs (hundreds worldwide)
  - Closest to users
  - Cache hot content
  - 1-10ms from most users

Layer 2: Regional / Origin Shield (20-50 locations)
  - Aggregates requests from multiple edge PoPs
  - Shields origin from repeated misses
  - Edge cache miss → try Shield before Origin
  - Shield cache hit ratio: 80-95% (dramatically reduces origin load)

Layer 3: Origin (your servers)
  - Only sees ~1-5% of requests (cache misses through all layers)
  - Can be a single global cluster or multi-region
```

Cache Key Design — Critical for CDN Efficiency:

```
Default cache key: URL (scheme + host + path + query string)
  GET https://cdn.apple.com/images/logo.png?v=3  → cached by full URL

Problem: Vary: Accept-Language header
  Same URL but user in French gets French content
  Without cache key variation: French user gets English content!

Cache key = URL + relevant request headers
  For internationalized content: URL + Accept-Language
  For A/B testing: URL + Experiment-Group cookie
  For auth'd content: URL + User-ID (PII risk — careful with edge caching!)
```

HTTP Caching Headers:

```
# Origin response headers that control CDN behavior

Cache-Control: public, max-age=86400, s-maxage=3600
  public     → CDN can cache (vs private = CDN must not cache)
  max-age    → browser cache duration (86400 = 24h)
  s-maxage   → CDN cache duration (3600 = 1h, overrides max-age for CDNs)

Cache-Control: no-store
  → Never cache (PII, financial data, per-user responses)

Cache-Control: no-cache
  → CDN must revalidate with origin on every request (ETag/Last-Modified)

Surrogate-Control: max-age=86400  (Fastly/Varnish specific header)
  → CDN-only directive, stripped before sending to browser

Vary: Accept-Encoding
  → Cache separate copies for gzip/brotli/uncompressed

CDN-Cache-Control: max-age=600  (Cloudflare specific)
  → Override s-maxage just for Cloudflare
```

Cache Invalidation — The Hardest Problem:

```
Option 1: TTL expiry
  → Content stale until TTL expires
  → 1-hour TTL = up to 1 hour of stale content after deploy

Option 2: Cache purge (URL-based)
  → CDN API: purge("https://cdn.apple.com/app/style.css")
  → Immediate invalidation
  → Must purge EVERY edge PoP (fan-out to hundreds of servers)

Option 3: Cache tags / surrogate keys
  → Tag responses with logical groupings
  → "article-123" tag: all URLs serving article 123
  → Single API call purges all tagged URLs across all PoPs
  → Fastly (Surrogate-Key), Cloudflare (Cache-Tag), Akamai (Fast Purge Tags)

Option 4: Versioned URLs (best for immutable assets)
  → /static/app-v1.2.3.js → /static/app-v1.2.4.js
  → Old URL keeps long TTL (CDN serves cached forever)
  → New URL populated on first request
  → No purge needed!
  → Perfect for JS, CSS, images with content-hash in filename
```

Edge Computing — Logic at the CDN:

```
// Cloudflare Worker: run code at the edge (50ms from any user)
addEventListener('fetch', event => {
  event.respondWith(handleRequest(event.request));
});

async function handleRequest(request) {
  const url = new URL(request.url);
  const country = request.cf.country;  // available from Cloudflare

  // A/B testing at edge: no origin roundtrip needed
  const experimentGroup = Math.random() < 0.5 ? 'A' : 'B';

  // Personalized redirect based on geography
  if (country === 'CN' && !url.hostname.includes('cn.')) {
    return Response.redirect('https://www.apple.com.cn' + url.pathname);
  }

  // Authentication at edge
  const token = request.headers.get('Authorization');
  if (!isValidToken(token)) {
    return new Response('Unauthorized', { status: 401 });
  }

  // Modify request before forwarding to origin
  const modifiedRequest = new Request(request, {
    headers: { ...request.headers, 'X-Edge-Country': country }
  });

  return fetch(modifiedRequest);
}
```

CDN for Dynamic Content (Dynamic Site Acceleration):

```
Problem: API responses are per-user → can't be cached
Solution: CDN still helps via:

1. TCP Connection Reuse
   User → CDN Edge (persistent connection, TLS already established)
   Edge → Origin (persistent, pre-warmed connection pool, shorter path)
   Result: 30-50% latency reduction on dynamic content

2. Protocol Optimization
   User → Edge: HTTP/3 (QUIC, no head-of-line blocking)
   Edge → Origin: HTTP/2 multiplexed, or HTTP/1.1 keep-alive

3. Route Optimization
   CDN uses private backbone (faster than public internet)
   Ookla Speedtest: public internet 100ms US→EU, Cloudflare Argo: 40ms

4. Compression at Edge
   Brotli compression at edge for origin that only supports gzip
```

CDN DDoS Mitigation:

```
Anycast absorbs volume:
  100Gbps attack → distributed across 100 PoPs = 1Gbps each (manageable)

Layer 3/4: Cloudflare Magic Transit (BGP-level, before TCP)
Layer 7: Rate limiting, bot detection, CAPTCHA challenges

Challenge pass:
  Suspicious IP → JS challenge (proves browser) → trusted for 24h
  Bot → fails JS challenge → blocked
  DDoS: 50M req/s → 99.9% fail challenge → only 50K reach origin
```

CDN Architecture for Apple App Store Example:

```
App binary download (2GB):
  - Versioned URL: /download/app-com.example.app-v3.2.1.ipa
  - Cache-Control: public, max-age=31536000, immutable
  - Served from CDN edge: 100% cache hit after first user downloads
  - Origin: only sees 1 request per version per PoP
  - Bandwidth savings: 99.9%+

App Store page (dynamic, personalized):
  - Cannot cache full page (personalized recommendations, purchase state)
  - Cache static assets (JS/CSS/images) with long TTL
  - Dynamic API response: no-store
  - TCP optimization via CDN for dynamic: 40ms savings

Metadata (app name, description, ratings):
  - Semi-dynamic: changes rarely
  - Cache with s-maxage=300, purge on app update
  - Surrogate-Key: "app-123" for instant purge on developer update
```

Multi-CDN Strategy:

```
Single CDN: SPOF if provider has outage
Multi-CDN: Route 50% to Cloudflare, 50% to Fastly

Traffic steering:
  - DNS-based: GeoDNS returns different CDN based on region
  - Anycast-based: both CDNs announce same IP, BGP routes to nearest
  - Real User Monitoring (RUM): measure actual user latency to each CDN
    → dynamically shift traffic to faster CDN for each region

Multi-CDN adds complexity:
  → Cache key consistency (same content on both)
  → Purge APIs for both CDNs on deploy
  → Analytics aggregation from both
```

Key Metrics:

Metric	*Target*	*How to Improve*
Cache Hit Ratio	>90%	Long TTLs, versioned URLs
TTFB (Time to First Byte)	<100ms	Edge caching, route optimization
Origin Request Rate	<10% of total	Cache-hit ratio improvement
Bandwidth savings	>80%	Static asset caching
Availability	99.999%	Multi-CDN, anycast

> ■ ***Apple Interview Tip:*** *"Apple's software updates (iOS, macOS, App Store) are the largest CDN workloads in the world. Versioned URLs with immutable cache headers mean the CDN serves billions of GB without origin load. For an interview: discuss cache key design (URL + Vary headers), invalidation strategy (tags for articles, versioned URLs for static assets), and multi-CDN for avoiding SPOF."*

Quick Reference: When to Use What

Concept	*Use When*
Gossip Protocol	Membership/topology in large clusters (Cassandra, DynamoDB)
Merkle Trees	Replica reconciliation, data integrity, blockchain proofs
Vector Clocks	Detecting concurrent writes, conflict resolution in eventual-consistent systems

Concept	*Use When*
Paxos	Strong consensus where you need safety guarantees (leader election, config stores)
Skip Lists	O(log N) sorted data with range queries; simpler than balanced BST
Tumbling Windows	Fixed-period aggregations (hourly sales, daily stats)
Sliding Windows	Continuous anomaly detection ("any 5-min window")
Session Windows	User-activity-based analytics (session duration, bounce rate)
Kubernetes	Container orchestration, microservices platform, stateful workloads
GeoDNS	Route users to nearest datacenter for latency reduction
CDN Edge Caching	Static assets, media files — versioned URLs + long TTL
CDN Dynamic Acceleration	APIs — TCP pre-warming, route optimization, compression

---

Part of the Apple Interview Preparation Series.

See also: systemdesignvol1.md (Staff), systemdesignvol2.md (Senior Staff), systemdesignvol3.md (Principal)